

Fundamentals of Deep Learning and Programming

Lecture 1: Introduction to Python Programming

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Python



Version: 3.10

Advantages

- **Easy** to learn,
- suitable for developing **deep learning SW**, **web** applications

Disadvantages

- **Slow** execution time
- **High memory** usage -> **unsuitable** for developing mobile applications

Python

Hello

0	1	2	3	4
-5	-4	-3	-2	-1

```
def add (a,b) :  
    return a+b  
print (add (2,5))
```

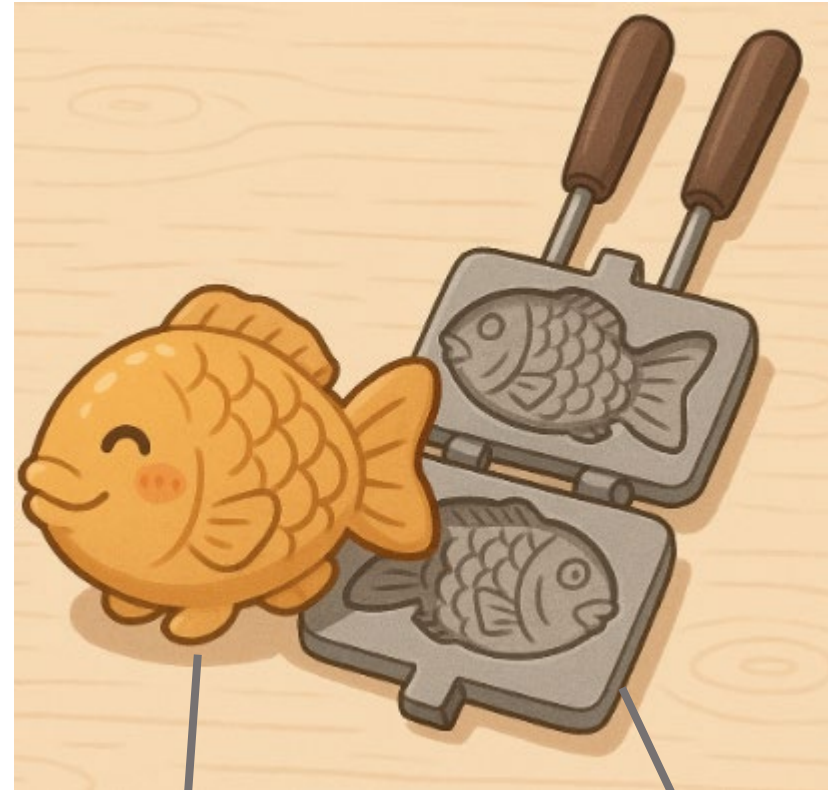
7

```
add = lambda a, b: a+b  
print (add (2,5))
```

7

Instance of class

```
class MoldFish:  
    pass  
  
goldFish = MoldFish()  
blueFish = MoldFish()
```



- **MoldFish** defines a class.
- **goldFish = MoldFish()** creates an **instance** of the MoldFish class and assigns it to the variable goldFish.

goldFish
(instance of class)

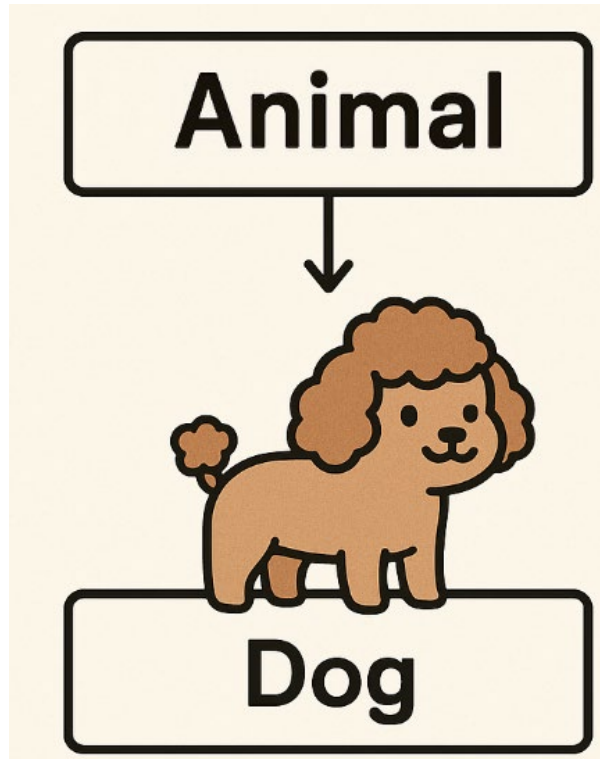
MoldFish
(class name)

Class Inheritance

```
# Parent class: Animal
class Animal:
    pass

# Child class: Dog inherits from Animal
class Dog(Animal):
    pass

# Create an instance of Dog
dog1 = Dog()
```



Dog **inherits** from Animal

Constructor

```
def __init__(self, ...):
```

```
class Dog:
    # The __init__ function is automatically called when an instance of the Dog class is created.
    def __init__(self, name, age):
        # self.name and self.age are attributes of the object.
        # These attributes are initialized with the values passed when the __init__ function is called
        self.name = name
        self.age = age
        print(f"A dog object with the name {self.name} has been created.")

    def bark(self):
        print(f"{self.name} barks mung mung !")
```

```
# Create instances of the Dog class
# At this point, __init__(self, "Buddy", 3) is automatically called.
dog1 = Dog("Kongi", 3)
dog2 = Dog("Baduki", 5)

# Access the object's attributes
print(f"dog1's name: {dog1.name}, age: {dog1.age}")
print(f"dog2's name: {dog2.name}, age: {dog2.age}")

# Call the method
dog1.bark()
```

```
(base) PS D:\dev\python\myTorch> python classDog.py
A dog object with the name Kongi has been created.
A dog object with the name Baduki has been created.
dog1's name: Kongi, age: 3
dog2's name: Baduki, age: 5
Kongi barks mung mung !
```

```

# Parent class: Animal
class Animal:
    # Constructor: defines common attributes
    def __init__(self, name, species):
        self.name = name          # instance attribute: name
        self.species = species    # instance attribute: species

    # Method: general animal sound
    def make_sound(self):
        print("Some generic animal sound")

    # Method: print basic information
    def info(self):
        print(f"{self.name} is a {self.species}")

# Child class: Dog inherits from Animal
class Dog(Animal):
    # Constructor: overrides the parent constructor
    def __init__(self, name, breed):
        # Call the parent constructor using super()
        super().__init__(name, species="Dog")
        self.breed = breed        # child-specific attribute: breed

    # Override the make_sound method
    def make_sound(self):
        print("Mung! Mung!")

    # Dog-specific method
    def fetch(self, item):
        print(f"{self.name} fetches the {item}")

# Create an instance of Dog
dog1 = Dog("Kongi", "Poodle")

# Access attributes
print(dog1.name)          # Kongi
print(dog1.species)       # Dog (inherited from Animal)
print(dog1.breed)         # Poodle (specific to Dog)

# Call methods
dog1.info()               # Kongi is a Dog (from Animal)
dog1.make_sound()         # Woof! Woof! (overridden method)
dog1.fetch("ball")        # Kongi fetches the ball (Dog's own method)

```

```

(base) PS D:\dev\python\myTorch> python animalDog.py
Kongi
Dog
Poodle
Kongi is a Dog
Mung! Mung!
Kongi fetches the ball

```